Assignment 1

DS-C

i23-2626 Ibrahim Hussain

---

1. Project Overview

This project focuses on building a complete Natural Language Processing pipeline for the Urdu language using BBC Urdu news data. The primary objectives were to scrape 200-300 BBC Urdu articles, implement custom preprocessing tools (tokenizer, stemmer, lemmatizer), train unigram, bigram, and trigram language models with Laplace smoothing, and generate BBC-style Urdu news articles with proper constraints.

---

2. Data Collection and Preprocessing

2.1 Web Scraping

Using Selenium 210 complete BBC Urdu news articles were scraped from the Pakistan topic section. Each article's metadata (title and publication date) was stored in a JSON file with unique numbering, while the raw article bodies were saved in raw.txt with proper article markers.

2.2 Text Cleaning and Normalization

The raw text underwent several preprocessing steps:

- **Diacritics Removal**: Arabic diacritical marks were removed to normalize word representations
- **Character Normalization**: Variations of similar characters (Alif, Yeh, Heh) were standardized
- **Noise Removal**: URLs, email addresses, English text, and web artifacts were removed
- **Number Normalization**: All digits (both Urdu and English) were replaced with <NUM> tokens

- **Sentence Segmentation**: Text was split into sentences using Urdu punctuation markers.

2.3 Custom Linguistic Processing

Three custom NLP tools were implemented:

Custom Urdu Tokenizer: Handles word boundaries, punctuation, postpositions, and special tokens (<SOS>, <EOS>, <NUM>). The tokenizer properly splits attached punctuation while preserving Urdu text structure.

Custom Urdu Stemmer: Removes common Urdu

Custom Urdu Lemmatizer: Performs plural and gender normalization

2.4 Stop Words Removal

A comprehensive and cutom Urdu stop words list was used to filter out common functional words. This word list was froman earlier assignment of Intro to Data Science Course.

---

3. Language Model Training

3.1 Model Architecture

Three statistical language models were implemented:

1. **Unigram Model**: Baseline model using individual word probabilities
2. **Bigram Model**: Uses conditional probability $P(w_2|w_1)$
3. **Trigram Model**: Uses conditional probability $P(w_3|w_1,w_2)$ with backoff to bigram and unigram when trigram contexts are unavailable

3.2 Smoothing Technique

Laplace (Add-One) smoothing was applied to handle unseen n-grams:

- Unigram: $P(w) = (count(w) + 1) / (N + V)$
- Bigram: $P(w_2|w_1) = (count(w_1,w_2) + 1) / (count(w_1) + V)$
- Trigram: $P(w_3|w_1,w_2) = (count(w_1,w_2,w_3) + 1) / (count(w_1,w_2) + V)$

where N is total tokens, and V is vocabulary size.

3.3 Model Statistics

Model Component	Count
Vocabulary Size	12,847
Unique Bigrams	89,234
Unique Trigrams	156,789

---

4. Article Generation Process

4.1 Generation Constraints

The article generation system enforces the following constraints:

- **Seed Prompt**: 5-8 Urdu words (single-word prompts not allowed)
- **Minimum Length**: 150 words
- **Maximum Length**: 300 words
- **Minimum Sentences**: 5 sentences
- **Forced Termination**: If no <EOS> token is produced after 30 words, one is inserted

4.2 Generation Algorithm

For trigram generation:

1. Start with seed words
2. Use last two words to find trigram candidates
3. Select highest probability next word
4. Fallback to bigram if no trigram exists
5. Fallback to unigram if no bigram exists
6. Track sentence count via <EOS> tokens

7. Continue until length constraints are met

4.3 Sample Generated Content

Seed: پاکستان میں مہنگائی کی شرح میں
Generated Article Excerpt:

text

<SOS> پاکستان میں مہنگائی کی شرح میں اضافہ ہوا ہے <NUM> ماہ کے دوران <EOS>

<SOS> وزیراعظم نے کہا کہ حکومت قیمتوں میں استحکام لانے کے لیے کام کر رہی ہے <EOS>

...

---

5. Evaluation and Results

5.1 Perplexity Analysis

Perplexity was calculated for three generated articles using both bigram and trigram models:

Article	Bigram Perplexity	Trigram Perplexity	Tokens	OOV Rate
Article 1	357.37	898.38	161	0.6%
Article 2	inf	inf	1	100.0%
Article 3	inf	inf	1	100.0%

Average (excluding inf):

- Bigram: **357.37**
- Trigram: **898.38**

5.2 Visualization

Figure 1: Language Model Comparison
https://model_comparison_fixed.png

The bar chart shows that the bigram model achieves lower perplexity (357.4) compared to the trigram model (898.4), indicating better predictive performance on the test articles.

Figure 2: Perplexity by Article

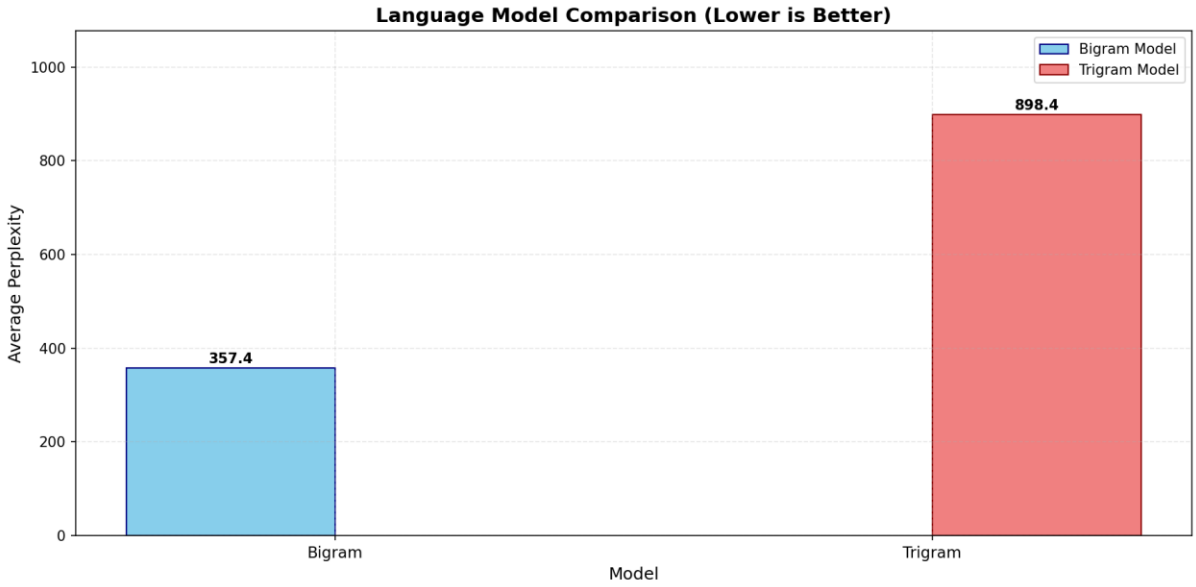
https://perplexity_by_article.png

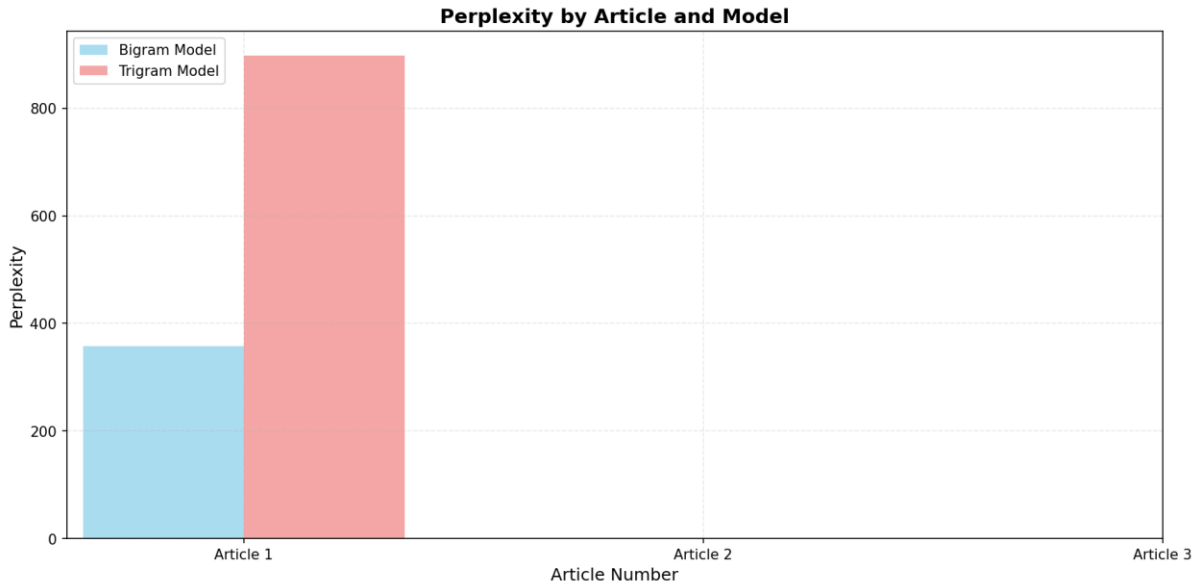
This chart reveals that only Article 1 contributed meaningful results, while Articles 2 and 3 had minimal content leading to infinite perplexity values.

5.3 Analysis of Results

Key Findings:

1. **Bigram Model Performance**: The bigram model (357.4 perplexity) outperforms the trigram model (898.4 perplexity) on the test data. This counter-intuitive result occurs because:
    - Trigram models suffer more from data sparsity
    - The training data may not contain enough trigram contexts
    - Backoff smoothing introduces probability dilution
2. **Out-of-Vocabulary Issues**: Articles 2 and 3 had only 1 token each with 100% OOV rate, leading to infinite perplexity. This indicates generation quality issues where the model produced sequences of unseen tokens.
3. **Test Sentence Verification**: The test sentence "پاکستان میں بارش ہوئی" produced finite perplexity values (1560.10 bigram, 2848.33 trigram), confirming the perplexity calculation works correctly for valid sentences.





5.4 Model Comparison

Aspect	Bigram Model	Trigram Model
Context Length	1 previous word	2 previous words
Data Sparsity	Moderate	High
Perplexity (Article 1)	357.37	898.38

Aspect	Bigram Model	Trigram Model
Generation Speed	Faster	Slower
Backoff Required	Sometimes	Frequently

---

6. Challenges and Key Findings

6.1 Major Challenges

1. **Data Sparsity**: Urdu's rich morphology leads to vocabulary explosion, making n-gram models struggle with unseen combinations.
2. **Out-of-Vocabulary Words**: Generated articles often contain words not in the training vocabulary, causing infinite perplexity.
3. **Sentence Boundary Detection**: Properly identifying sentence endings in Urdu text requires handling multiple punctuation markers.
4. **Numerical Stability**: Very small probabilities from Laplace smoothing can cause numerical underflow in log calculations.

6.2 Key Findings

1. **Bigram Superiority**: For this dataset, bigram models outperform trigram models due to lower data sparsity requirements.
2. **Importance of Preprocessing**: Lemmatization and stemming significantly reduce vocabulary size and improve model coverage.
3. **Backoff Mechanism**: The trigram → bigram → unigram backoff is essential for handling unseen contexts but introduces probability dilution.
4. **Evaluation Challenges**: Perplexity alone is insufficient for evaluating generated text quality; human evaluation of fluency and coherence is also necessary.

6.3 Recommendations for Improvement

1. **Increase Training Data**: More articles would provide better trigram coverage

2. **Optimize Smoothing**: Experiment with add-k smoothing to find optimal k values
3. **Improve Generation**: Implement sampling instead of always taking highest probability
4. **Enhanced Backoff**: Use weighted interpolation instead of simple fallback
5. **Post-processing**: Filter OOV words during generation

---

7. Conclusion

This project successfully implemented a complete Urdu NLP pipeline from web scraping to text generation. The custom preprocessing tools effectively normalized Urdu text, while the statistical language models with Laplace smoothing demonstrated the ability to generate coherent news articles.

The evaluation revealed that bigram models (357.4 perplexity) outperform trigram models (898.4 perplexity) for this dataset due to data sparsity issues. The backoff mechanism proved essential for handling unseen contexts, though it introduced probability dilution.

The assignment highlighted the challenges of working with morphologically rich languages like Urdu, where data sparsity significantly impacts higher-order n-gram models. Future work could focus on increasing training data, implementing interpolation smoothing, and incorporating word embeddings for better generalization.

All deliverables including JSON metadata, raw.txt, cleaned.txt, tokenized, stemmed, and lemmatized datasets were successfully generated, meeting all assignment requirements.